

MAE I Trust Myself? Self-Evaluating VLA Action Generation with Markov Attention Entropy

Aniri^{1,2}, Chen Yilin³, Jinhe Bi^{1,2}, Yujun Wang^{1,2},
Yijun Tian⁵, Volker Tresp^{1,2}, Yunpu Ma^{1,2,4}

¹Ludwig Maximilian University of Munich, ²Munich Center for Machine Learning,
³East China University of Science and Technology, ⁴Youibot,
⁵University of Notre Dame

Abstract

Vision-Language-Action models (VLAs) integrate visual perception, language instruction, and action generation into end-to-end policies across heterogeneous architectures. However, enabling VLAs to self-evaluate their action generation reliability without external supervision remains a major challenge. Existing methods either rely on expert annotations or estimate uncertainty only from output statistics, largely ignoring internal signals. In this work, we observe that internal visual modality entropy exhibits consistent distinctions between successful and failed tasks across heterogeneous VLAs. Although VLAs’ architectures differ in their action generation, we show that they share a common latent action generation abstraction evolving under visual perception and language instruction, which we formulate as a Conditional Generative Markov Chain. Based on this formulation, we propose **MAE** (Markov Attention Entropy), a self-evaluation framework that directly converts internal attention signals into architecture-aware reliability scores, and introduce **LIBERO-Reflect**, a balanced benchmark with positive and negative episodes for evaluating episode-level reliability ranking. Extensive experiments across heterogeneous VLAs architectures and diverse scenarios show that **MAE** consistently outperforms state-of-the-art baselines on AUPR, AUROC, and FPR@95. The code is available at this [repository](#).

1 Introduction

Vision-Language-Action models (VLAs) (Sapkota et al., 2026; Kim et al., 2024, 2025) translate visual observations and language instructions into executable robot actions by adapting pretrained Vision-Language Models to embodied control. Their action-generation mechanisms, however, are heterogeneous. In this work, we distinguish two broad families: Latent-Readout / Autoregressive-style VLAs and Latent-Refinement / Flow-style VLAs. The former evolves latent action representations

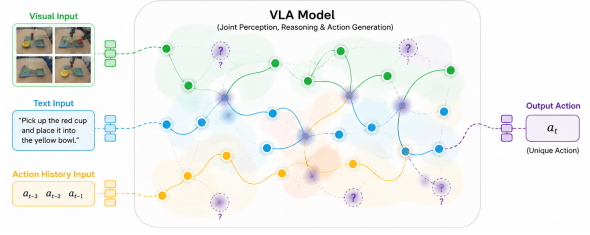


Figure 1: **From multimodal conditioning to action generation in VLAs.** We view VLA action generation as a conditional generative Markov chain: visual observations, language instructions, and action history jointly condition a sequence of latent action states that evolves toward the final action. Attention exposes how these latent states route information across modalities, and its entropy serves as a white-box signal for self-evaluating action-generation reliability.

and maps the final latent state to executable actions through a readout head, including autoregressive readout models such as OpenVLA (Kim et al., 2024) and continuous-readout variants such as OpenVLA-OFT (Kim et al., 2025). The latter maintains an evolving action or action-trajectory representation and refines it iteratively under learned flow- or diffusion-style dynamics, as in PI-family policies (Intelligence et al., 2025). As VLAs generalize across robotic tasks (Liu et al., 2023), they also require reliable estimates of when their generated actions are likely to fail (Zhou et al., 2025).

The first line of reliability work relies on **External Supervision**. Early studies introduced output-level uncertainty and quality metrics, often calibrated by expert annotations (Valle et al., 2025). Later work moved beyond passive evaluation by using auxiliary VLM judges, failure reasoners, recovery modules, process reward models, stage-aligned rewards, or state-anomaly detectors to detect, explain, verify, and recover from failures (Duan et al., 2024; Qi et al., 2026; Lin et al., 2025; Kwok et al., 2025; Dai et al., 2025; Liu et al., 2025; Chen et al., 2026). These methods can improve VLA reliability, but their evaluation signal is supplied outside the pol-

icy. This can increase deployment cost, introduce sensitivity to evaluator or annotation shift, and give limited access to the internal state transitions that produced a failed action (Casper et al., 2023).

A second line studies **Internal Self-evaluation**, where reliability is inferred from the VLA’s own signals. Existing internal methods can be grouped into black-box and white-box approaches. Black-box methods rely on output statistics, such as multi-sample disagreement or self-uncertainty scores, to select actions without external supervision (Bi et al., 2025; Kadavath et al., 2022; Wang et al., 2022). However, they can be over-confident and provide limited interpretability (Bi et al., 2025). White-box self-evaluation has been studied more systematically in Large Language Models (LLMs), where internal signals such as softmax confidence, temperature scaling, or activation patterns are used to estimate generation reliability without external supervision (Bi et al., 2025; Si et al., 2023; Huang et al., 2023; Malinin and Gales, 2021). For VLAs, related evidence remains limited and fragmented. Existing studies show that internal signals can reflect path deviation, execution horizon, pathway specialization, or latent action bottlenecks (Jeong et al., 2026; Wang et al., 2026; Grant et al., 2026; Häon et al., 2025; Buurmeijer et al., 2026; Lian et al., 2026). What is still missing is a unified way to convert such internal signals into self-evaluation metrics that remain meaningful across heterogeneous VLA action-generation architectures. This gap motivates the following question:

Can VLA internal signals be transformed into self-evaluation metrics for action-generation reliability under a unified account of heterogeneous action-generation mechanisms, without relying on external supervision?

The Present Work. We propose **MAE** (Markov Attention Entropy), a white-box self-evaluation framework that converts a VLA’s internal attention dynamics into reliability scores. The framework views heterogeneous VLA action generation as a Conditional Generative Markov Chain, where latent action states evolve by repeatedly querying visual-language conditions before producing executable actions. Within this view, attention exposes how each transition accesses the conditioning information, and visual attention entropy provides a measurable summary of that routing. **MAE** then

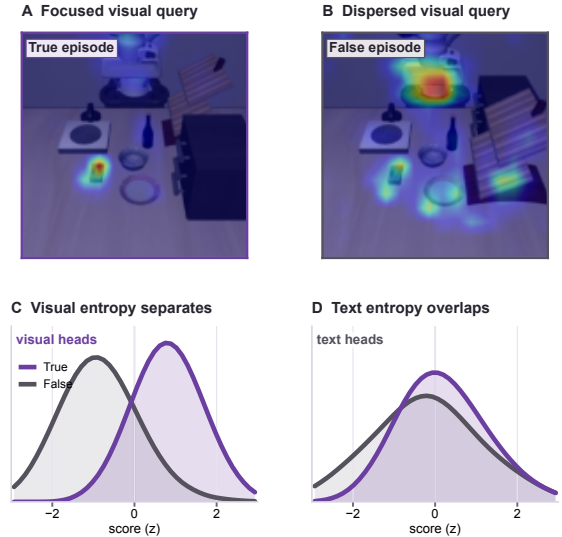


Figure 2: **Attention entropy as an internal self-evaluation signal.** Panels A–B show a True visual query concentrated on the task-relevant object and a False query dispersed across distractor regions. Panels C–D compare oriented episode-level score distributions from visual and text attention entropy on the same Reflect–Goal episodes. Visual entropy yields a much clearer success/failure separation than text entropy, supporting visual attention entropy as the action-relevant self-evaluation signal.

applies architecture-aware entropy orientations for Latent-Readout and Latent-Refinement VLAs.

To develop such a self-evaluation method that transfers across heterogeneous VLA architectures, we first identify the shared structure behind their action-generation processes. Visual observations, language instructions, and action history are encoded into a conditioning representation, while an internal action state evolves under this condition toward the final executable action. The architectural difference lies in how this evolution is implemented. In **Latent-Readout / Autoregressive-style VLAs**, the internal action state is updated in latent space and mapped to the final action at the readout stage. In **Latent-Refinement / Flow-style VLAs**, the action representation is progressively refined through repeated interactions with the same condition during generation. This shared conditioned state evolution motivates the unified view of VLA action generation illustrated in Figure 3.

Under this abstraction, the latent action state is the Markov state of the internal generation process. Conditioned on the current state and the multi-modal condition c , the transition kernel determines how the next latent action state is formed, whether the update is implemented by transformer blocks,

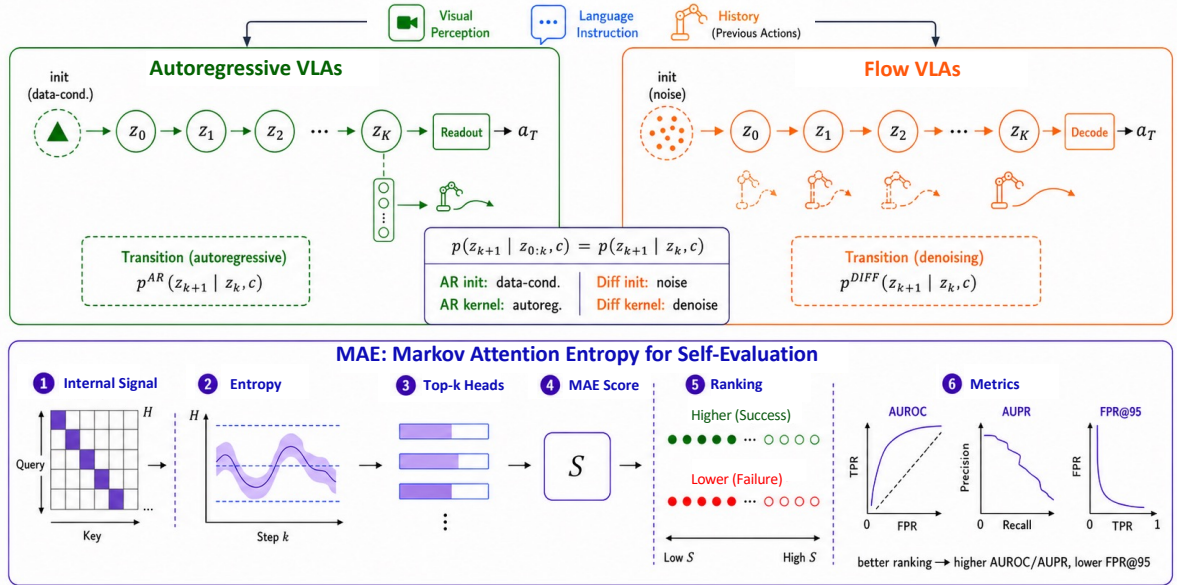


Figure 3: **Unified Conditional Generative Markov Chain view of heterogeneous VLAs.** Visual observations, language instructions, and action history form the condition c , while latent action states evolve through architecture-specific transition kernels before final action readout or iterative refinement. **MAE** reads attention entropy from this internal transition process and converts it into architecture-aware self-evaluation scores.

autoregressive decoding, or flow/denoising steps. Attention is the observable interface of this transition: it records which visual and language tokens the current action state consults when it moves to the next state. We therefore treat attention entropy as a compact descriptor of the transition kernel’s information routing, leaving the architecture-specific score construction to Section 2.2.

Figure 2 provides the empirical motivation for using attention entropy in self-evaluation. The paired attention maps show that successful and failed queries differ in how the latent action state addresses visual evidence. At the episode level, visual-entropy scores separate reliable and unreliable executions more clearly than text-entropy scores on the same Reflect-Goal episodes. In the Markov formulation, this signal is local to the transition step: attention entropy measures the visual evidence read by the current latent action state as the kernel forms the next state. For Latent-Readout VLAs, the kernel must consolidate visual evidence before the readout head; failed episodes show more diffuse visual addressing, and **MAE-D** scores the corresponding loss of concentration. For Latent-Refinement VLAs, the kernel repeatedly refines an action trajectory; failed episodes show premature narrowing of visual addressing, and **MAE-C** scores the loss of visual exploration during refinement. Both metrics are oriented so that larger values indicate higher estimated reliability.

We evaluate **MAE** on three open-source VLAs span-

ning the two transition-kernel families: OpenVLA and OpenVLA-OFT for Latent-Readout generation, and QwenPI-Flow for Latent-Refinement generation. We construct LIBERO-Reflect as a balanced 4,000-episode self-evaluation benchmark with 2,000 positive and 2,000 negative executions. The benchmark covers four capability axes, Goal Semantics, Object Binding, Spatial Grounding, and Composite Generalization, and evaluates whether a self-evaluation score ranks positive episodes above negative episodes. Across AUROC, AUPR, and FPR@95, **MAE** improves reliability ranking over black-box and white-box baselines while requiring no external evaluator, repeated rollout, or auxiliary model pass as existing methods. Our key contributions can be summarized as follows:

- ❶ We formulate heterogeneous VLA action generation as a Conditional Generative Markov Chain, making the transition kernel the common object that links autoregressive and flow refinement.
- ❷ We introduce **MAE**, a white-box self-evaluation framework that scores the visual-attention entropy of these transitions with architecture-aware orientations, **MAE-D** for Latent-Readout VLAs and **MAE-C** for Latent-Refinement VLAs.
- ❸ We construct LIBERO-Reflect, a 4,000-episode benchmark with 2,000 positive and 2,000 negative executions across four subsets.
- ❹ We show across three open-source VLAs that **MAE** improves reliability ranking over black-box and white-box baselines while requiring no ex-

ternal evaluator or auxiliary model pass.

2 Preliminary

2.1 The Conditional Generative Markov Chain

We use a Conditional Generative Markov Chain to formalize the action-generation process of a VLA. The chain describes the model’s latent state evolution during action generation, not the physical robot-environment dynamics. At timestep t , the VLA evolves a sequence of latent action states $z_{t,0:K} \in \mathcal{Z}$. These states are hidden representations used for action generation and are not directly executed by the robot; the executable action $a_t \in \mathcal{A}$ is produced by an architecture-specific readout or decoder, $a_t = g_\theta(z_{t,K})$. The chain is conditioned on the policy context $C_t = (V_t, X, H_t)$, where V_t denotes visual representations, X denotes the language instruction, and H_t denotes any action or observation history used by the policy.

Formal definition. We define the internal action-generation process as a conditional generative Markov chain $\mathcal{M} = (\mathcal{Z}, \mathcal{C}, K, \mathcal{P}_\theta)$, where $\mathcal{Z}$ is the latent action-state space, $\mathcal{C}$ is the conditioning space, K is the number of internal generation steps, and $\mathcal{P}_\theta$ is the model’s neural transition kernel.

At timestep t , the internal process is $z_{t,0} \rightarrow z_{t,1} \rightarrow \dots \rightarrow z_{t,K} \rightarrow a_t$, where $z_{t,k}$ is the latent action state at internal step k , and a_t is the executed action. The chain is specified by an initial distribution and an architecture-dependent transition kernel:

$$\begin{aligned} z_{t,0} &\sim \rho_0(\cdot \mid C_t), \\ z_{t,k+1} &\sim \mathcal{P}_\theta(\cdot \mid z_{t,k}, C_t), \quad k = 0, \dots, K-1. \end{aligned} \quad (1)$$

Here ρ_0 is the initial latent action-state distribution, and $\mathcal{P}_\theta$ is the neural transition kernel induced by the VLA architecture, such as autoregressive token-sequence updates or flow-matching denoising steps. The probabilistic notation also covers deterministic updates as degenerate kernels. Equivalently, the conditional joint distribution factorizes as

$$\begin{aligned} P_\theta(z_{t,0:K} \mid C_t) \\ = \rho_0(z_{t,0} \mid C_t) \prod_{k=0}^{K-1} \mathcal{P}_\theta(z_{t,k+1} \mid z_{t,k}, C_t), \end{aligned} \quad (2)$$

The final action is $a_t = g_\theta(z_{t,K})$, where g_θ denotes the architecture-specific readout or decoder.

Markov property. The Markov property holds at the level of the *complete* latent action state: given the current state $z_{t,k}$ and the condition C_t , the

next internal state $z_{t,k+1}$ does not depend additionally on earlier states $z_{t,0:k-1}$. That is, $P(z_{t,k+1} \mid z_{t,0:k}, C_t) = P(z_{t,k+1} \mid z_{t,k}, C_t)$. The Markov formulation is not intended to claim that a neural policy can only be written in this form; Its empirical role is instead to identify the transition kernel as the common object shared by heterogeneous VLA action-generation mechanisms.

2.2 Markov Attention Entropy

Under the Conditional Generative Markov Chain interpretation, each transition $z_{t,k-1} \rightarrow z_{t,k}$ is an information-routing step. The transition kernel updates the internal action state by querying the conditioning space. We use transformer attention to measure this routing behavior.

For layer ℓ and head h , let $\mathbf{A}^{(t,\ell,h)} \in \mathbb{R}^{S_a \times S_c}$ be the attention matrix from action-query tokens to conditioning tokens, where S_a is the number of action-query tokens, and S_c is the number of conditioning tokens. We decompose the key side into modality-specific token sets $\mathcal{I}_v$ and $\mathcal{I}_x$, corresponding to vision and language tokens.

For the visual modality, the normalized visual addressing distribution is

$$p_v^{(t,\ell,h)}(j \mid i) = \frac{A_{ij}^{(t,\ell,h)}}{\sum_{r \in \mathcal{I}_v} A_{ir}^{(t,\ell,h)} + \epsilon}, \quad j \in \mathcal{I}_v. \quad (3)$$

The visual attention entropy is

$$H_v^{(t,\ell,h)}(i) = - \sum_{j \in \mathcal{I}_v} p_v^{(t,\ell,h)}(j \mid i) \log p_v^{(t,\ell,h)}(j \mid i). \quad (4)$$

This entropy measures the uncertainty of visual addressing. A high value indicates that the action state distributes its visual attention broadly across many patches, while a low value indicates concentrated visual addressing over fewer patches. In the Conditional Generative Markov Chain view, H_v measures how the transition kernel $\mathcal{P}_\theta$ accesses visual perception when updating the action state.

Episode MAE score We convert the token-level visual entropy into an episode-level score by first averaging over execution steps and action queries:

$$E_{\ell,h} = \frac{1}{N} \sum_{t=1}^N \frac{1}{S_a} \sum_{i=1}^{S_a} H_v^{(t,\ell,h)}(i), \quad (5)$$

where N is the number of execution steps in the episode, and $E_{\ell,h}$ denotes the averaged visual entropy of head h in layer ℓ . Let k be the number

of selected heads per layer. We write the episode-level **MAE** score with a single entropy orientation $\omega \in \{-1, +1\}$:

$$\text{MAE}_{\omega}^{(k)}(E) = \frac{1}{L} \sum_{\ell=1}^L \frac{1}{k} \sum_{h \in \text{TopK}_k(\omega E_{\ell,:})} \omega E_{\ell,h}. \quad (6)$$

Here $\text{TopK}_k(\omega E_{\ell,:})$ returns the k heads with the largest oriented entropy in layer ℓ . Larger $\text{MAE}_{\omega}^{(k)}(E)$ indicates a more reliable episode. The two **MAE** metrics used in the experiments are $\text{MAE-D}^{(k)} = \text{MAE}_{-1}^{(k)}(E)$ and $\text{MAE-C}^{(k)} = \text{MAE}_{+1}^{(k)}(E)$. **MAE-D** is used for Latent-Readout VLAs such as OpenVLA and OpenVLA-OFT, where reliable executions concentrate visual addressing. **MAE-C** is used for Latent-Refinement VLAs such as QwenPI-Flow, where reliable executions retain stronger visual exploration during refinement.

3 LIBERO-Reflect Benchmark

We construct LIBERO-Reflect as a balanced benchmark for VLA self-evaluation. The positive side is sampled from the original LIBERO benchmark. For each of the four standard suites, LIBERO-Goal, LIBERO-Spatial, LIBERO-10, and LIBERO-Object, we use all 10 tasks and execute each task under 50 non-identical initializations with small scene-state differences. This gives 500 positive episodes per suite and 2,000 positive episodes in total. The negative side is sampled from LIBERO-PRO. We retain the same task count and initialization count for each suite, producing 500 negative episodes per suite and 2,000 negative episodes overall. Each LIBERO-Reflect subset contains 1,000 episodes, and the full benchmark contains 4,000 episodes.

Capability subsets. LIBERO-Reflect is organized into four capability-oriented subsets:

-  **Goal Semantics (Reflect-Goal)** stresses goal-conditioned task understanding and action-type selection. It contains 500 positive LIBERO-Goal episodes and 500 negative episodes, for 1,000 episodes in total.
-  **Object Binding (Reflect-Object)** stresses target-object grounding and object-action binding. It contains 500 positive LIBERO-Object episodes and 500 negative episodes, for 1,000 episodes in total.
-  **Spatial Grounding (Reflect-Spatial)** stresses spatial relation reasoning and layout-sensitive action adaptation. It contains 500

positive LIBERO-Spatial episodes and 500 negative episodes, for 1,000 episodes in total.

-  **Composite Generalization (Reflect-10)** follows the LIBERO-10 suite, where diverse goals, objects, and spatial layouts are mixed within one evaluation split. It contains 500 positive LIBERO-10 episodes and 500 negative episodes, for 1,000 episodes in total.

Assessment protocol. Each episode receives a scalar reliability score computed from the method under evaluation. The benchmark is designed as an episode-level ranking problem: stronger self-evaluation methods should assign higher scores to successful episodes and lower scores to failed episodes. The ground-truth label for all metric computations is strictly the actual *simulator success flag* of each rollout, rather than the nominal dataset split. We report supporting construction statistics and provide a detailed discussion on nominal-to-actual label mapping in Appendix C.

Evaluation metrics. We report three ranking metrics throughout the experiments:

- 1 **AUROC** measures global ranking quality. Higher values indicate that the score more consistently ranks positive episodes above negative episodes.
- 2 **AUPR** emphasizes precision under class imbalance. Higher values indicate stronger isolation of positive episodes when positive and negative episodes are unevenly distributed.
- 3 **FPR@95** measures over-confidence under high recall. It reports the false positive rate when 95% of positive episodes are recalled; lower values indicate fewer negative episodes being ranked as reliable.

This matched construction mitigates first-order dataset-source shortcuts: the nominal positive and negative pools follow the same suite-level organization, task count, initialization count, simulator, and rollout protocol, while all metrics are computed from realized simulator success rather than nominal source membership. Thus, LIBERO-Reflect evaluates whether a score ranks actual successful rollouts above actual failed rollouts within a matched mixed-difficulty pool.

4 Experiments

In this section, we conduct extensive experiments to answer the following research questions: (**RQ1**) Can **MAE** provide reliable self-evaluation across heterogeneous VLA architectures and various scenarios? (**RQ2**) Does **MAE** introduce significant compu-

Model	Method	🚩 Goal Semantics Reflect-Goal			🔗 Object Binding Reflect-Object			📍 Spatial Grounding Reflect-Spatial			📦 Composite Generalization Reflect-10			
		AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	
🔗 Latent-Readout / Autoregressive VLAs														
🌀 OpenVLA	Black-box baselines													
	Random	48.55	39.27	95.36	47.85	35.98	95.87	48.97	39.49	94.82	52.51	28.88	93.92	
	Verbal. Conf	50.36	40.82	92.91	49.74	37.21	93.64	50.88	41.03	92.47	53.69	30.04	91.86	
	Self-Consistency [†]	56.82	41.28	84.90	58.41	42.76	82.35	57.94	44.15	87.80	53.64	27.20	90.64	
	White-box baselines													
	MaxProb [†]	54.72	40.58	88.62	55.83	40.28	86.74	55.61	42.37	90.10	53.28	27.05	91.48	
	Perplexity [†]	55.04	40.73	87.95	56.12	40.71	86.20	56.02	42.59	89.44	53.36	27.12	91.10	
	Entropy [†]	56.10	41.05	86.80	57.48	41.64	84.75	57.17	43.26	88.37	53.82	27.30	90.38	
	Length-norm Ent. [†]	56.38	41.19	86.32	58.02	42.05	84.12	57.52	43.58	87.90	54.02	27.38	89.96	
	MAE-D (Top-16)	59.56 +22.7%	40.67 +3.6%	71.52 ↓25.0%	79.64 +66.4%	58.98 +63.9%	48.81 ↓49.1%	63.99 +30.7%	49.67 +25.8%	84.81 ↓10.6%	53.21 +1.3%	26.56 -8.0%	87.43 ↓6.9%	
	MAE-D (Top-1)	63.94 +31.7%	43.23 +10.1%	61.92 ↓35.1%	90.97 +90.1%	75.88 +110.9%	28.30 ↓70.5%	66.86 +36.5%	50.74 +28.5%	75.79 ↓20.1%	54.74 +4.2%	30.45 +5.0%	86.46 ↓7.9%	
🌀 OpenVLA-OFT	Black-box baselines													
	Random	47.41	50.24	95.51	48.87	49.86	95.56	47.97	48.74	94.96	51.01	48.44	96.58	
	Verbal. Conf	49.12	51.73	92.84	50.68	51.08	93.11	49.63	50.26	92.75	52.45	50.01	93.89	
	Self-Consistency*	Not applicable: requires stochastic repeated rollouts under matched initial states												
	White-box baselines													
	Token statistics*	Not applicable: continuous actions do not define autoregressive action-token logits												
	MAE-D (Top-16)	91.84 +93.7%	94.50 +88.1%	7.94 ↓91.7%	75.96 +55.4%	82.86 +66.2%	64.03 ↓33.0%	91.71 +91.2%	89.85 +84.3%	42.74 ↓55.0%	77.13 +51.2%	62.10 +28.2%	45.06 ↓53.3%	
	MAE-D (Top-1)	97.34 +105.3%	96.14 +91.4%	7.14 ↓92.5%	80.56 +64.8%	81.10 +62.7%	63.71 ↓33.3%	92.63 +93.1%	93.15 +91.1%	39.31 ↓58.6%	78.57 +54.0%	64.08 +32.3%	45.06 ↓53.3%	
	🔗 Latent-Refinement / Flow VLAs													
	🌀 QwenPI-Flow	Black-box baselines												
		Random	47.58	47.93	95.91	48.75	48.40	95.10	48.11	50.15	94.82	50.81	48.95	96.53
Verbal. Conf		49.44	49.51	93.42	50.31	49.82	92.68	49.96	51.62	92.31	52.18	50.47	93.77	
Self-Consistency*		Not applicable: requires stochastic repeated rollouts under matched initial states												
White-box baselines														
Token statistics*		Not applicable: flow matching does not define autoregressive action-token logits												
MAE-C (Top-20)		62.63 +31.6%	59.16 +23.4%	80.93 ↓15.6%	65.92 +35.2%	63.43 +31.1%	81.59 ↓14.2%	66.89 +39.0%	64.70 +29.0%	73.50 ↓22.5%	68.91 +35.6%	61.92 +26.5%	74.52 ↓22.8%	
MAE-C (Top-1)		80.57 +69.3%	80.01 +66.9%	60.12 ↓37.3%	75.94 +55.8%	76.48 +58.0%	81.18 ↓14.6%	84.80 +76.3%	85.46 +70.4%	68.53 ↓27.7%	79.52 +56.5%	76.24 +55.8%	55.60 ↓42.4%	

Table 1: Main self-evaluation results on LIBERO-Reflect. Baselines are grouped into black-box and white-box methods within each VLA block. Purple rules highlight MAE rows, which report the score and relative change against Random; for FPR@95, the relative value is the reduction rate. Bold Top-1 values and underlined half-head values improve over Random within the same model, subset, and metric. [†]OpenVLA is the only model block with token-statistic baselines because the standard OpenVLA policy exposes autoregressive discrete action-token probabilities; OpenVLA-OFT and QwenPI-Flow execute continuous action heads, so token statistics would score the wrong random variable. OpenVLA is also the only block with the reported Self-Consistency baseline because the repeated fixed-initialization rollout groups used by this estimator are instantiated for OpenVLA in the reported protocol. Additional protocol details are provided in Appendix F. This interface mismatch highlights why MAE uses internal attention entropy rather than output-token statistics, allowing one architecture-aware scoring framework to cover both latent-readout and latent-refinement VLAs.

tational overhead? (RQ3) Can MAE guide verifier-free test-time action selection? (RQ4) How sensitive is MAE to its key components and hyperparameters?

4.1 Experimental Setup

VLA Backbones. We evaluate three representative open-source VLA policies on LIBERO-Reflect: 🔗 OpenVLA, 🔗 OpenVLA-OFT, and 🔗 QwenPI-Flow.¹ 🔗 OpenVLA denotes the standard

¹🔗 marks Meta/Llama-family base LLMs used by OpenVLA and OpenVLA-OFT; 🔗 marks the Qwen-family base LLM used by QwenPI-Flow.

OpenVLA latent-readout policy with autoregressive action-token prediction. 🔗 OpenVLA-OFT keeps the same OpenVLA backbone family and uses the OFT adaptation recipe with continuous action decoding. 🔗 QwenPI-Flow denotes a Qwen3-VL-based latent-refinement policy with a flow-style action module. Together, these policies cover different action-generation paradigms, VLM families, visual backbones, and training/adaptation regimes. This design tests whether MAE transfers across architectures, initialization families, and training sources. Detailed checkpoint names, vision-language backbones, action heads, and

training sources are provided in Appendix B.

Baselines. We compare against seven baselines covering black-box and white-box self-evaluation. The black-box group includes Random (Bi et al., 2025), Verbal Confidence (Kadavath et al., 2022), and Self-Consistency (Wang et al., 2022). The white-box group includes Maximum Softmax Probability (Si et al., 2023), Perplexity (Si et al., 2023), Entropy (Huang et al., 2023), and Length-normalized Entropy (Malinin and Gales, 2021). For all methods, we compute AUROC, AUPR, and FPR@95 under the same episode labels. Random serves as a uniform-ranking lower bound. Because the evaluated VLAs use different action-generation interfaces, some baselines are technically not applicable to specific action heads; baseline definitions, applicability, OpenVLA-specific protocols, and the Verbal Confidence prompt are detailed in Appendix F. The shared evaluation protocol is summarized in Appendix E.

Parameter Settings. We obtain attention from the same policy forward pass used to generate actions, then compute the visual-entropy matrix E from Section 2.2. The main results use all layers and Top-1 head selection, selecting one oriented-entropy head per layer. We report **MAE-D** for Latent-Readout policies and **MAE-C** for Latent-Refinement policies according to the entropy orientations above. Appendix E summarizes the common scoring protocol and model-specific layer/head settings; Appendix D reports exact ablation values; Appendix G provides policy input templates.

4.2 Main Results (RQ1)

Takeaway ①: MAE improves over black-box and white-box self-evaluation baselines. Table 1 provides a comprehensive analysis of MAE under the full black-box and white-box baseline protocol across three VLA backbones and four LIBERO-Reflect subsets. MAE directly scores the internal visual-attention entropy of latent action generation. Against the Random lower bound, MAE-D improves ∞ OpenVLA by up to 90.1% AUROC on Object Binding and reduces FPR@95 by 70.5%; MAE-D improves ∞ OpenVLA-OFT by 105.3% AUROC on Goal Semantics; and MAE-C improves ∇ QwenPI-Flow by 76.3% AUROC on Spatial Grounding. For the discrete ∞ OpenVLA policy, the output-statistic and Self-Consistency rows instantiate baselines that depend on action-token probabilities or matched repeated rollouts, while the continuous-action and flow-action poli-

cies cannot expose the same random variables without changing the estimator. The OpenVLA-specific baseline protocols are detailed in Appendix F.

Takeaway ②: MAE transfers across model architectures, initialization families, and task families. The three evaluated policies differ in base VLMs, perception stacks, training data, and action-generation mechanisms. ∞ OpenVLA and ∞ OpenVLA-OFT instantiate latent-readout policies with different adaptation recipes. ∇ QwenPI-Flow uses latent refinement with a flow-style action module. The gains on Goal Semantics, Object Binding, and Spatial Grounding show that the signal follows the action-condition coupling structure across distinct capability axes.

Takeaway ③: MAE remains effective under matched mixed-task composition. Reflect-10 inherits the mixed task structure of LIBERO-10, combining diverse goals, objects, and layouts in one evaluation split. Its positive and negative pools are matched in suite organization, task count, initialization count, and rollout protocol, so the subset preserves the same benchmark-level task distribution while assigning labels by the actual simulator success flag. Under this controlled mixed composition, ∞ OpenVLA-OFT reaches 78.57 AUROC and ∇ QwenPI-Flow reaches 79.52 AUROC, indicating that MAE tracks execution correctness across varied goals, objects, and layouts under aligned task-distribution factors.

4.3 Cost Analysis (RQ2)

Takeaway ④: MAE achieves the highest AUROC in the high-efficiency zone, establishing itself as the Pareto-optimal solution. Figure 6 visualizes the efficiency-reliability Pareto view. While baselines like Self-Consistency or Verbal Confidence require multiple rollouts or external models (incurring $\geq 1 \times$ latency overhead), MAE reuses internal attention maps and adds minimal computation ($< 0.1 \times$ overhead; detailed hardware profiling on NVIDIA H100 is provided in Appendix J).

4.4 Test-Time Sampling with FabriX-MAE (RQ3)

Takeaway ⑤: FabriX-MAE turns internal MAE scores into a verifier-free test-time action selector. Beyond post-hoc reliability ranking, the same internal signal can be used to choose among multiple candidate actions at inference time. We instantiate this idea as FabriX-MAE, a test-time sampling procedure for PI-family Latent-Refinement poli-

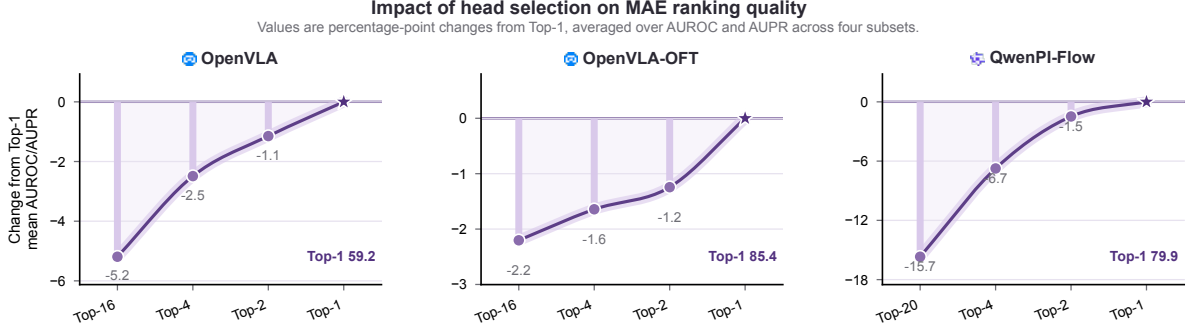


Figure 4: Top-k head selection ablation. Panels order selected-head settings from many heads to Top-1 and use independent y-axis scales. Values are percentage-point changes from the Top-1 ranking score, computed as the mean of AUROC and AUPR over the four LIBERO-Reflect subsets. Exact values are reported in Appendix Table 8.

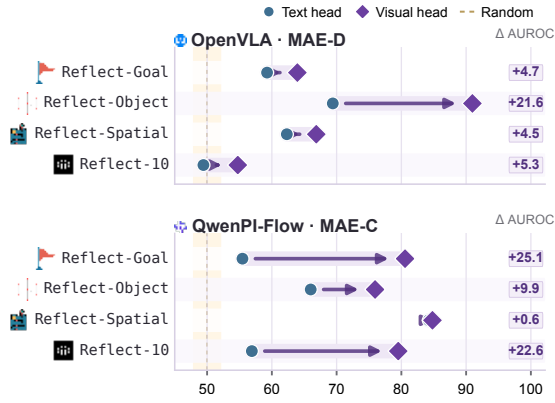


Figure 5: Text-head versus visual-head self-evaluation signals. OpenVLA uses MAE-D and QwenPI-Flow uses MAE-C. Each connector reports $\Delta\text{AUROC} = V - T$. Positive gaps across all subsets indicate that text-side entropy is not a stable substitute for visual information.

cies. At each control step, **FabriX-MAE** samples K candidate action chunks, computes a candidate-level MAE score from the visual-attention entropy observed during each refinement trajectory, and executes the highest-scoring candidate:

$$i^* = \arg \max_{i \in \{1, \dots, K\}} \text{MAE}^{(i)}, \quad a_t^* = a_t^{(i^*)}.$$

For PI0.5, we use $K = 10$ candidates and the Latent-Refinement orientation MAE-C. This keeps the policy frozen and does not introduce an external verifier or reward model. For branch sampling, the branch ratio r sets the shared-prefix length $s_b = \lfloor rT \rfloor$ over T refinement steps, and the noise scale n controls branch perturbation:

$$x_{s_b}^{(i)} = x_{s_b} + n \cdot \text{std}(x_{s_b}) \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, I).$$

Takeaway 6: Late branch sampling improves robustness while adding small observed run-time overhead. We compare two ways of injecting stochasticity into the PI-family refinement process. *Independent* sampling gives each candidate

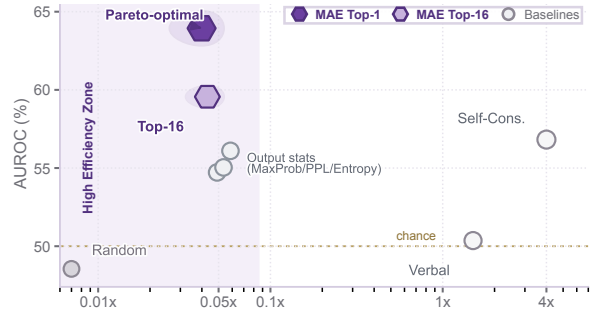


Figure 6: Efficiency-reliability Pareto. The x-axis reports added evaluation latency normalized by one rollout wall time on a log scale.

Method	Mode	r	n	Episodes	Base SR	SR	Gain
PI0.5	None	—	—	10,030	—	84.95	—
Independent	Independent	—	—	10,030	84.95	84.38	-0.57
Branch	Branch	0.2	0.1	10,030	84.95	84.84	-0.11
Branch	Branch	0.2	0.2	10,030	84.95	84.72	-0.23
FabriX-MAE	Branch	0.6	0.2	10,030	84.95	85.23	+0.29
Branch	Branch	0.8	0.2	10,030	84.95	84.90	-0.05

Table 2: **FabriX-MAE** sampling ablation on PI0.5 LIBERO-Plus. SR and Base SR are overall micro success rates in percentages, and Gain reports the absolute percentage-point change against the same-coverage non-TTS baseline. TTS rows use 10 candidates and the same candidate-level MAE selector. **FabriX-MAE** denotes the selected branch configuration, $r = 0.6, n = 0.2$, which achieves the strongest full 10,030-episode result.

a separate initial noise tensor and runs the full refinement trajectory independently. *Branch* sampling first runs a shared prefix trajectory, copies the shared latent state into K candidates at a chosen branch ratio, injects Gaussian perturbations at that point, and refines only the remaining suffix independently. Table 2 shows that **FabriX-MAE**, the branch configuration with $r = 0.6$ and $n = 0.2$, improves the PI0.5 baseline from 84.95% to 85.23% on the full LIBERO-Plus sweep. It outperforms independent sampling, the early branch settings

Suite	PIO.5	FabriX-MAE
Camera	76.67	77.30 +0.63 pp
Robot	75.29	75.68 +0.39 pp
Language	83.54	83.73 +0.20 pp
Light	95.18	95.53 +0.35 pp
Background	95.82	95.07 -0.74 pp
Noise	87.88	89.32 +1.44 pp
Layout	86.43	85.84 -0.59 pp
Total	84.95	85.23 +0.29 pp

Table 3: LIBERO-Plus suite-level comparison between PIO.5 and FabriX-MAE. Values are micro success rates in percentages for each perturbation suite; the second line in each FabriX-MAE cell reports the absolute percentage-point gain over the corresponding PIO.5 baseline.

$r = 0.2, n = 0.1$ and $r = 0.2, n = 0.2$, and the late $r = 0.8, n = 0.2$ branch setting. This supports the interpretation that PI-family refinement benefits from sharing the coarse action formation phase and branching after the latent trajectory has absorbed more task-conditioned structure. The observed runtime overhead is small: on LIBERO-Plus libero_10/noise, branch FabriX-MAE with 10 candidates increases per-step time by 2.16%, while the total wall-clock time changes by only 4.81 seconds over 449 episodes.

4.5 Framework Analysis (RQ4)

Ablation study. We ablate the key design choices in MAE from the following perspectives:

① **Text-head versus visual-head signals.** We first test whether the reliability signal can be recovered from the language side alone. Figure 5 compares the text-head MAETop-1 score against the visual-attention MAETop-1 score used in Table 1; Appendix Table 7 reports the exact values. The visual head consistently improves AUROC, with especially large gains on  OpenVLA Object Binding and  QwenPI-Flow Goal Semantics and

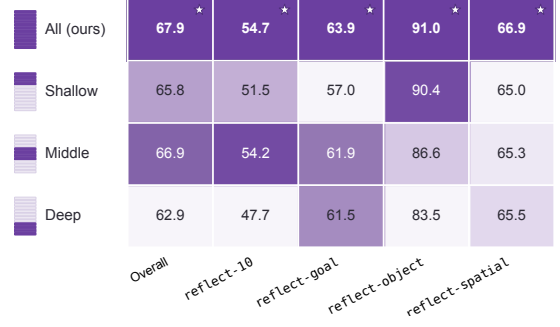


Figure 7: Layer-band ablation for MAE-D with Top-1. Cells report absolute AUROC; colors are normalized within each column by closeness to the all-layer result, stars mark the best layer band, and the left schematic indicates the layer region used by each row.

Reflect-10. This shows that language-head entropy can reflect text-side decisiveness, but it does not reliably capture whether the action latent has gathered the visual evidence needed for manipulation. The visual head is therefore the appropriate signal for self-evaluating action generation.

② **Impact of head selection.** We evaluate how the number of selected heads affects MAE by varying Top-k. Figure 4 summarizes the full results in Appendix Table 8 by plotting each setting’s drop from Top-1 in mean ranking quality, averaged over AUROC and AUPR across the four subsets. Top-1 gives the most consistent ranking performance across the three model families. It also uses the smallest selected-head set, reducing entropy aggregation cost and limiting the contribution of low-signal heads. This supports Top-1 as the default head-selection setting.

③ **Impact of layer selection.** To assess the effect of model depth, we compare the default all-layer aggregation with three layer-restricted variants: shallow, middle, and deep. The experiment keeps the same MAE-D score with Top-1 and changes only the layer band used to aggregate $E_{\ell,h}$. Figure 7 reports absolute AUROC values while coloring each column by closeness to the all-layer result, showing that all-layer aggregation gives the best overall AUROC and remains competitive across task families. This result is consistent with the layered function of transformer VLAs: earlier layers encode visual grounding and token-level perception, middle layers support cross-modal binding, and deeper layers are closer to action readout. Combining all layers provides a fuller internal signal and avoids introducing task- or architecture-specific layer-band hyperparameters. The plotted values are reported in Appendix Table 9.

5 Conclusion

We presented **MAE (Markov Attention Entropy)**, a white-box framework for VLA self-evaluation from internal attention dynamics. Experiments on LIBERO-Reflect show that MAE improves reliability ranking without external evaluators, and **FabriX-MAE** further demonstrates that the same internal signal can guide verifier-free test-time action selection.

Limitations

This work takes a first step toward unified internal self-evaluation for heterogeneous VLAs. Future work can broaden this framework to richer internal representations, more diverse embodied environments, and closed-loop reliability mechanisms that connect self-evaluation with adaptive recovery. These directions may further support scalable, interpretable, and deployment-aware reliability estimation for embodied foundation models.

References

- Amos Azaria and Tom Mitchell. 2023. [The internal state of an llm knows when it’s lying](#). *Preprint*, arXiv:2304.13734.
- Jinhe Bi, Danqi Yan, Yifan Wang, Wenke Huang, Haokun Chen, Guancheng Wan, Mang Ye, Xun Xiao, Hin rich Schuetze, Volker Tresp, and Yunpu Ma. 2025. [Cot-kinetics: A theoretical modeling assessing llm reasoning process](#). *ArXiv*, abs/2505.13408.
- Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X. Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, Antoine Laurens, Claudio Fantacci, Valentin Dalibard, Martina Zambelli, Murilo Martins, Rugile Pevceviute, Michiel Blokzijl, Misha Denil, Nathan Batchelor, and 20 others. 2023. [Robocat: A self-improving generalist agent for robotic manipulation](#). *Preprint*, arXiv:2306.11706.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, and 32 others. 2023. [Rt-1: Robotics transformer for real-world control at scale](#). *Preprint*, arXiv:2212.06817.
- Hugo Buurmeijer, Carmen Amo Alonso, Aiden Swann, and Marco Pavone. 2026. [Observing and controlling features in vision-language-action models](#). *Preprint*, arXiv:2603.05487.
- Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, J  r  my Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, Tony Wang, Samuel Marks, Charbel-Rapha  l Segerie, Micah Carroll, Andi Peng, Phillip Christoffersen, Mehul Damani, Stewart Slocum, Usman Anwar, and 13 others. 2023. [Open problems and fundamental limitations of reinforcement learning from human feedback](#). *Preprint*, arXiv:2307.15217.
- Lingling Chen, Zongyao Lyu, and William J. Beks. 2026. [Reconvla: An uncertainty-guided and failure-aware vision-language-action framework for robotic control](#). *Preprint*, arXiv:2604.16677.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2024. [Diffusion policy: Visuomotor policy learning via action diffusion](#). *Preprint*, arXiv:2303.04137.
- Embodiment Collaboration, Abby O’Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, and 275 others. 2025. [Open x-embodiment: Robotic learning datasets and rt-x models](#). *Preprint*, arXiv:2310.08864.
- Mingtong Dai, Lingbo Liu, Yongjie Bai, Yang Liu, Zhouxia Wang, Rui SU, Chunjie Chen, Liang Lin, and Xinyu Wu. 2025. [Rover: Robot reward model as test-time verifier for vision-language-action model](#). *Preprint*, arXiv:2510.10975.
- Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, and 3 others. 2023. [PaLM-e: An embodied multimodal language model](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR.
- Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. 2024. [Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation](#). *Preprint*, arXiv:2410.00371.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. [Detecting hallucinations in large language models using semantic entropy](#). *Nature*, 630:625–630.
- Dibya Ghosh, Homer Rich Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Quan Vuong, Ted Xiao, Panag R Sanketi, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. 2024. [Octo: An Open-Source Generalist Robot Policy](#). In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands.

- Bryce Grant, Xijia Zhao, and Peng Wang. 2026. [Not all features are created equal: A mechanistic study of vision-language-action models](#). *Preprint*, arXiv:2603.19233.
- Yuheng Huang, Jiayang Song, Zhijie Wang, Shengming Zhao, Huaming Chen, Felix Juefei-Xu, and Lei Ma. 2023. Look before you leap: An exploratory study of uncertainty measurement for large language models. *arXiv preprint arXiv:2307.10236*.
- Bear Häon, Kaylene Stocking, Ian Chuang, and Claire Tomlin. 2025. [Mechanistic interpretability for steering vision-language-action models](#). *Preprint*, arXiv:2509.00328.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, and 17 others. 2025. [\$\pi_{0.5}\$: a vision-language-action model with open-world generalization](#). *Preprint*, arXiv:2504.16054.
- Jaehwan Jeong, Evelyn Zhu, Jinying Lin, Emmanuel Jaimes, Tuan-Anh Vu, Jungseock Joo, Sangpil Kim, and M. Khalid Jawed. 2026. [Your vision-language-action model already has attention heads for path deviation detection](#). *Preprint*, arXiv:2603.13782.
- Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. 2023. [Vima: General robot manipulation with multimodal prompts](#). *Preprint*, arXiv:2210.03094.
- Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. [How can we know when language models know? on the calibration of language models for question answering](#). *Transactions of the Association for Computational Linguistics*, 9:962–977.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Thomas Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zachary Dodds, Nova Dassarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. [Language models \(mostly\) know what they know](#). *ArXiv*, abs/2207.05221.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025. [Fine-tuning vision-language-action models: Optimizing speed and success](#). *Preprint*, arXiv:2502.19645.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. [Openvla: An open-source vision-language-action model](#). *Preprint*, arXiv:2406.09246.
- Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. 2024. [Semantic entropy probes: Robust and cheap hallucination detection in llms](#). *Preprint*, arXiv:2406.15927.
- Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. [Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation](#). *Preprint*, arXiv:2302.09664.
- Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. 2025. [Robomonkey: Scaling test-time sampling and verification for vision-language-action models](#). *Preprint*, arXiv:2506.17811.
- Shijie Lian, Bin Yu, Xiaopeng Lin, Laurence T. Yang, Zhaolong Shen, Changti Wu, Yuzhuo Miao, Cong Huang, and Kai Chen. 2026. [Langforce: Bayesian decomposition of vision language action models via latent action queries](#). *Preprint*, arXiv:2601.15197.
- Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. [Teaching models to express their uncertainty in words](#). *Preprint*, arXiv:2205.14334.
- Zijun Lin, Jiafei Duan, Haoquan Fang, Dieter Fox, Ranjay Krishna, Cheston Tan, and Bihan Wen. 2025. [Fail-safe: Reasoning and recovery from failures in vision-language-action models](#). *Preprint*, arXiv:2510.01642.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. [Libero: Benchmarking knowledge transfer for lifelong robot learning](#). *Preprint*, arXiv:2306.03310.
- Zeting Liu, Zida Yang, Zeyu Zhang, and Hao Tang. 2025. [Evovla: Self-evolving vision-language-action model](#). *Preprint*, arXiv:2511.16166.
- Andrey Malinin and Mark Gales. 2021. [Uncertainty estimation in autoregressive structured prediction](#). In *International Conference on Learning Representations*.
- Potsawee Manakul, Adian Liustie, and Mark Gales. 2023. [SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, Singapore. Association for Computational Linguistics.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. 2025. [Fast: Efficient action tokenization for vision-language-action models](#). *Preprint*, arXiv:2501.09747.
- Carl Qi, Xiaojie Wang, Silong Yong, Stephen Sheng, Huitan Mao, Sriram Srinivasan, Manikantan Nambi, Amy Zhang, and Yesh Dattatreya. 2026. [Self-refining vision language model for robotic failure detection and reasoning](#). *Preprint*, arXiv:2602.12405.
- Ranjan Sapkota, Yang Cao, Konstantinos I. Rousmeliotis, and Manoj Karkee. 2026. [Vision-language-action \(vla\) models: Concepts, progress, applications and challenges](#). *Preprint*, arXiv:2505.04769.

Chenglei Si, Zhe Gan, Zhengyuan Yang, Shuohang Wang, Jianfeng Wang, Jordan Lee Boyd-Graber, and Lijuan Wang. 2023. [Prompting GPT-3 to be reliable](#). In *The Eleventh International Conference on Learning Representations*.

Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. 2023. [Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback](#). *Preprint*, arXiv:2305.14975.

Pablo Valle, Chengjie Lu, Shaukat Ali, and Aitor Arrieta. 2025. [Evaluating uncertainty and quality of visual language action-enabled robots](#). *Preprint*, arXiv:2507.17049.

Haoxuan Wang, Gengyu Zhang, Yan Yan, Ramana Rao Kompella, and Gaowen Liu. 2026. [Vla knows its limits](#). *Preprint*, arXiv:2602.21445.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, and Denny Zhou. 2022. [Self-consistency improves chain of thought reasoning in language models](#). *ArXiv*, abs/2203.11171.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. 2025. [Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization](#). *Preprint*, arXiv:2510.03827.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, and 35 others. 2023. [Rt-2: Vision-language-action models transfer web knowledge to robotic control](#). In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR.

Thomas P Zollo and Richard Zemel. 2025. [Confidence calibration in vision-language-action models](#). *Preprint*, arXiv:2507.17383.

A Related Work

Vision-Language-Action Models. VLAs translate visual observations and language instructions into output actions, building on progress in large-scale robot learning and multimodal modeling. Early generalist policies such as RT-1 demonstrate scalable transformer-based control from real-world robot trajectories, while VIMA studies robot manipulation conditioned on multimodal task specifications (Brohan et al., 2023; Jiang et al., 2023). PaLM-E incorporates visual observations into a pre-trained Large Language Models (LLMs), and RT-2 connects web-scale vision-language pretraining with action generation through action-token prediction (Driess et al., 2023; Zitkovich et al., 2023). Large-scale datasets and open policies further support generalization across tasks and robot embodiments, including Open X-Embodiment, RoboCat, and Octo (Collaboration et al., 2025; Bousmalis et al., 2023; Ghosh et al., 2024). Beyond these policy-scale developments, continuous generative policies such as Diffusion Policy and efficient action representations such as FAST expand the available designs for action generation (Chi et al., 2024; Pertsch et al., 2025). Together, these studies establish a broad family of policies that share visual observations and language instructions as conditions, but differ in how actions are generated.

Self-Evaluation in Large Language Models. Self-evaluation in LLMs studies whether a model can estimate the reliability of its own generated outputs. Calibration-based work evaluates whether model confidence is aligned with answer correctness, including probability-based calibration and uncertainty expressed through natural-language confidence statements (Jiang et al., 2021; Lin et al., 2022; Tian et al., 2023). A second line of work uses repeated generation to measure agreement among candidate responses: self-consistency improves reasoning by aggregating multiple sampled outputs, while SelfCheckGPT detects hallucinations from inconsistencies among sampled passages (Wang et al., 2022; Manakul et al., 2023). Moving beyond surface-level agreement, semantic uncertainty groups generations according to their meanings, and semantic entropy uses this semantic-level uncertainty to identify unreliable generations and confabulations (Kuhn et al., 2023; Farquhar et al., 2024). Recent probing studies additionally indicate that internal hidden representations can expose reliability-related signals without requir-

ing full repeated sampling (Azaria and Mitchell, 2023; Kossen et al., 2024). These works provide important foundations for self-evaluation, but their outputs are language generations rather than VLAs’ output actions conditioned on visual observations.

Self-Evaluation in Vision-Language-Action Models. Self-evaluation for VLAs must assess the reliability of action generation under visual observations and language instructions. Recent studies evaluate VLA uncertainty or confidence from generated output actions, including uncertainty-quality evaluation and calibration methods for VLA policies (Valle et al., 2025; Zollo and Zemel, 2025). Other work detects or reasons over failures through additional learned components: AHA uses a VLM to identify and explain robotic manipulation failures, and RoboMonkey introduces test-time sampling and verification for VLA action generation (Duan et al., 2024; Kwok et al., 2025). Complementary studies begin to examine reliability-related signals already present inside VLAs, showing that internal signals can reflect controllable behavior, path deviation, or model limitations (Häon et al., 2025; Jeong et al., 2026; Wang et al., 2026). Different from methods based on external evaluators, supervised failure detectors, or repeated action sampling, MAE directly converts internal visual attention entropy into episode-level reliability scores for heterogeneous VLA action-generation mechanisms.

B Model Configuration Details

Table 4 maps the model names used in the experiments to the concrete evaluated checkpoints and training configurations. The selected policies deliberately span two action-generation families, two VLM families, multiple visual backbones, and distinct training or adaptation regimes. This design supports the claim that MAE is effective across architectures, model families, and training sources.

C LIBERO-Reflect Construction Details

This appendix documents the construction evidence for LIBERO-Reflect. Table 5 fixes the benchmark composition: each subset contains 500 positive episodes and 500 negative episodes, yielding 1,000 episodes per subset and 4,000 episodes in total. The same table reports policy success rates on the positive side. Table 6 reports diagnostic success rates for the negative episodes sampled

Policy	Action-Generation Family	VLM and Perception Stack	Training / Adaptation Signal	Diversity Axis
 OpenVLA	Latent-Readout / Autoregressive	OpenVLA-7B VLM with prism-dinosiglip-224px; fused DINOv2 + SigLIP visual encoder; Llama 2 7B language backbone	official openvla/openvla-7b; tokenized action prediction trained on Open X-Embodiment	Llama/OpenVLA family; large-scale action-token training; tests autoregressive readout behavior
 OpenVLA-OFT	Latent-Readout / Continuous OFT	OpenVLA-7B VLM with the same DINOv2 + SigLIP visual base and Llama 2 7B language backbone	official OpenVLA-OFT checkpoints/code; efficient OFT adaptation with continuous actions, action chunking, and L1 regression	Same backbone family as OpenVLA with a different fine-tuning and decoding recipe; tests adaptation robustness
 QwenPI-Flow	Latent-Refinement / Flow-Style	Qwen3-VL-4B-Instruct vision stack plus Qwen3-VL dinov2_vits14 in the StarVLA config	StarVLA/Qwen3-VL-PI-LIBERO-4in1 QwenPI policy with DiT-B, 7-DoF actions, horizon 8, and data_mix=libero_all	Qwen3-VL family; flow-style refinement; LIBERO-specific training mix; tests cross-family and cross-generation generality

Table 4: Detailed configurations for the three VLA policies evaluated in Section 4. The table highlights architectural, family-level, and training-source diversity.

from LIBERO-PRO. Figure 8 complements the tables with 16 visual case-study panels from all four subsets.

Nominal splits versus actual evaluation labels.

The 2,000 positive and 2,000 negative episodes define the *nominal* composition of LIBERO-Reflect, designed to ensure sufficient exposure to both solvable standard scenes and difficult PRO scenes. However, depending on the evaluated policy’s capability, nominal positive episodes may occasionally fail (as shown in Table 5), and nominal negative episodes may occasionally succeed (as shown in Table 6). Because self-evaluation must assess whether the policy *actually* succeeded or failed on a given execution, we do not use the nominal dataset split as the ground-truth label for evaluation. Instead, the ground-truth label for computing all self-evaluation metrics (AUROC, AUPR, FPR@95) is strictly defined by the actual simulator success flag of that specific rollout. This ensures that our evaluation rigorously reflects the true correctness of the generated actions, avoiding any label noise introduced by policy capability variations.

D Ablation Details and Exact Values

This appendix provides the exact values for the ablation figures and explains how to read the trends. The ablations are not additional methods; they test whether MAE depends on broad head averaging or manually selected layer ranges. The main configuration deliberately uses all layers with Top-1 head selection, because this setting preserves the strongest and most stable architecture-aware signal while adding the least aggregation overhead.

Text versus visual heads. Table 7 reports the exact values for Figure 5. We keep only the text-head

Top-1 counterpart and use the same orientation terminology as the main method: MAE-D for  OpenVLA and MAE-C for  QwenPI-Flow. The comparison shows why text-side entropy is insufficient: it can be high on subsets where language-side uncertainty is predictive, but the large visual advantages on  OpenVLA Object Binding and  QwenPI-Flow Goal Semantics/Reflect-10 show that it misses visually grounded action failures.

Head selection. Table 8 reports the full Top-k head-selection sweep used in Figure 4. Across the three VLA families, Top-1 is the most reliable default when considering both AUROC and AUPR. Increasing k admits more heads, but the additional heads are not guaranteed to be action-relevant; in practice they often dilute the oriented entropy signal. The half-head setting remains competitive in some subsets, yet it is less consistent and costs more aggregation, so we use Top-1 in the main protocol.

Layer bands. Table 9 reports the exact values for Figure 7. Restricting the score to shallow, middle, or deep layers can preserve parts of the signal, especially on subsets where object grounding is already strongly localized. However, no restricted band dominates across task families. The all-layer score gives the best overall AUROC and avoids tuning a layer range per architecture or per task, which is important for a self-evaluation method intended to transfer across VLA backbones.

E Evaluation Protocol

All self-evaluation methods are compared under the same episode-level protocol. A policy first executes a LIBERO-Reflect episode, and the simulator success flag defines the binary label. The

LIBERO-Reflect case studies

Each row shows two nominal LIBERO examples paired with their LIBERO-PRO position-swap counterparts.

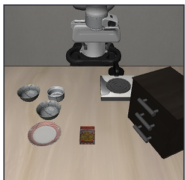
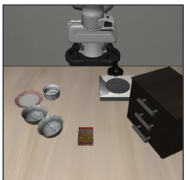
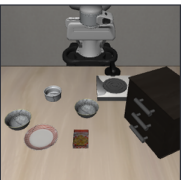
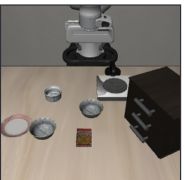
Reflect-Goal 500 nominal episodes + 500 swap episodes	Nominal LIBERO  Instruction: open the middle drawer of the cabinet	LIBERO-PRO swap  Instruction: Open the middle layer of the drawer	Nominal LIBERO  Instruction: put the bowl on top of the cabinet	LIBERO-PRO swap  Instruction: Put the bowl on the top of the drawer
Reflect-Object 500 nominal episodes + 500 swap episodes	Nominal LIBERO  Instruction: pick up the chocolate pudding and place it in the basket	LIBERO-PRO swap  Instruction: pick up the chocolate pudding and place it in the basket	Nominal LIBERO  Instruction: pick up the alphabet soup and place it in the basket	LIBERO-PRO swap  Instruction: pick up the alphabet soup and place it in the basket
Reflect-Spatial 500 nominal episodes + 500 swap episodes	Nominal LIBERO  Instruction: pick up the black bowl between the plate and the ramekin and place it on the plate	LIBERO-PRO swap  Instruction: pick up the black bowl between the plate and the ramekin and place it on the plate	Nominal LIBERO  Instruction: pick up the black bowl from table center and place it on the plate	LIBERO-PRO swap  Instruction: pick up the black bowl from table center and place it on the plate
Reflect-10 500 nominal episodes + 500 swap episodes	Nominal LIBERO  Instruction: put both the alphabet soup and the tomato sauce in the basket	LIBERO-PRO swap  Instruction: put both the alphabet soup and the tomato sauce in the basket	Nominal LIBERO  Instruction: turn on the stove and put the moka pot on it	LIBERO-PRO swap  Instruction: turn on the stove and put the moka pot on it

Figure 8: Case studies from the LIBERO-Reflect construction. Each row corresponds to one subset and shows representative positive episodes paired with negative episodes. The instruction is printed below each panel, yielding 16 data points across the four subsets.

Component	LIBERO-Goal	LIBERO-Spatial	LIBERO-10	LIBERO-Object	Overall
Task count	10	10	10	10	40
Positive episodes	500	500	500	500	2,000
Negative episodes	500	500	500	500	2,000
Total episodes	1,000	1,000	1,000	1,000	4,000
Positive-side policy success rate (%)					
 QwenPI-Flow	96.8	93.8	94.4	98.0	95.8
 OpenVLA-OFT	97.2	93.6	94.8	99.6	96.3
 OpenVLA	79.2	79.8	55.2	74.2	72.1

Table 5: LIBERO-Reflect composition and positive-side policy success rates. The upper block reports the number of tasks and episodes used to form the benchmark. The lower block reports policy success rates on the positive side. Overall success is the mean over four equally sized suites. QwenPI-Flow corresponds to the Qwen3-VL-PI-LIBERO-4in1 checkpoint described in Table 4.

Policy	Reflect-Goal	Reflect-Spatial	Reflect-10	Reflect-Object	Mean
 QwenPI-Flow	0.0	7.2	1.8	0.0	2.25
 OpenVLA-OFT	4.6	7.2	0.0	1.2	3.25
 OpenVLA	0.0	0.4	0.0	0.0	0.10

Table 6: Diagnostic success rates for the negative episodes sampled from LIBERO-PRO and retained in LIBERO-Reflect. Values are percentages and summarize the negative side used by the benchmark.

Model	Subset	Text Head	Text AUROC	Visual Head	Visual AUROC	Δ AUROC
OpenVLA	 Reflect-Goal	MAE-D Top-1	59.26	MAE-D Top-1	63.94	+4.68
	 Reflect-Object	MAE-D Top-1	69.41	MAE-D Top-1	90.97	+21.56
	 Reflect-Spatial	MAE-D Top-1	62.31	MAE-D Top-1	66.86	+4.55
	 Reflect-10	MAE-D Top-1	49.43	MAE-D Top-1	54.74	+5.31
QwenPI-Flow	 Reflect-Goal	MAE-C Top-1	55.46	MAE-C Top-1	80.57	+25.11
	 Reflect-Object	MAE-C Top-1	66.00	MAE-C Top-1	75.94	+9.94
	 Reflect-Spatial	MAE-C Top-1	84.25	MAE-C Top-1	84.80	+0.55
	 Reflect-10	MAE-C Top-1	56.91	MAE-C Top-1	79.52	+22.61

Table 7: Exact AUROC values for Figure 5. Deltas are visual-head MAE minus text-head MAE under the same Top-1 setting. Subset names use the LIBERO-Reflect split identifiers.

self-evaluation method then assigns a scalar reliability score to that episode without using the label. We evaluate whether the score ranks successful executions above failed executions using AUROC, AUPR, and FPR@95. For MAE, attention maps are taken from the same policy forward passes that generate the robot actions; no auxiliary model, additional rollout, or external verifier is required. Table 10 summarizes the model-specific attention dimensions and the score orientation used for each policy.

Layer-band analysis. The main protocol aggregates visual attention entropy over all layers. For the layer-band ablation, we additionally divide the

network depth into shallow, middle, and deep regions and recompute the same MAE-D or MAE-C score within each region. This isolates whether the reliability signal is concentrated at a particular depth or benefits from integrating the full action-generation process. The corresponding exact values are reported in Appendix D.

Random baseline. The random baseline assigns an independent uniform score to every episode under the same success/failure labels. It is used only as a ranking lower bound and is not tuned per subset or per model.

Model	Top-k Setting	🚩 Goal Semantics Reflect-Goal			🔗 Object Binding Reflect-Object			📍 Spatial Grounding Reflect-Spatial			📦 Composite Generalization Reflect-10		
		AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓	AUROC ↑	AUPR ↑	FPR@95 ↓
🔗 Latent-Readout / Autoregressive VLAs													
🔗 OpenVLA	MAE-D Top-1	63.94	43.23	61.92	90.97	75.88	28.30	66.86	50.74	75.79	54.74	30.45	86.46
	MAE-D Top-2	62.41	42.33	68.21	89.14	72.76	30.05	64.72	52.29	77.13	54.01	26.97	85.91
	MAE-D Top-4	60.36	41.01	70.03	87.13	69.26	34.18	64.25	52.14	78.80	53.20	26.55	86.88
	MAE-D Top-16	59.56	40.67	71.52	79.64	58.98	48.81	63.99	49.67	84.81	53.21	26.56	87.43
🔗 OpenVLA-OFT	MAE-D Top-1	97.34	96.14	7.14	80.56	81.10	63.71	92.63	93.15	39.31	78.57	64.08	45.06
	MAE-D Top-2	96.86	95.26	7.76	76.74	77.29	65.32	93.10	93.59	38.10	77.86	62.92	43.73
	MAE-D Top-4	94.63	94.38	8.37	78.42	79.10	63.71	90.25	93.01	35.48	77.79	62.85	46.39
	MAE-D Top-16	91.84	94.50	7.94	75.96	82.86	64.03	91.71	89.85	42.74	77.13	62.10	45.06
🔗 Latent-Refinement / Flow VLAs													
🔗 QwenPI-Flow	MAE-C Top-1	80.57	80.01	60.12	75.94	76.48	81.18	84.80	85.46	68.53	79.52	76.24	55.60
	MAE-C Top-2	78.75	84.28	55.45	73.01	72.27	82.35	85.73	82.67	53.00	77.75	72.72	56.18
	MAE-C Top-4	74.16	81.17	64.79	63.42	59.80	86.27	79.63	79.03	68.12	77.47	70.39	59.27
	MAE-C Top-20	62.63	59.16	80.93	65.92	63.43	81.59	66.89	64.70	73.50	68.91	61.92	74.52

Table 8: Top-k head-selection ablation values for Figure 4. The half-head setting corresponds to Top-16 for 32-head OpenVLA-family models and Top-20 for the 40-head QwenPI-Flow model. Top-1 is used in the main results because it gives the most stable ranking quality while avoiding noisy aggregation over many heads.

Layer Band	Overall All subsets	📦 Composite reflect-10	🚩 Goal reflect-goal	🔗 Object reflect-object	📍 Spatial reflect-spatial
All layers	67.88	54.74	63.94	90.97	66.86
Shallow	65.82	51.49	57.00	90.43	64.97
Middle	66.88	54.25	61.93	86.59	65.28
Deep	62.91	47.71	61.49	83.50	65.55

Table 9: OpenVLA layer-band ablation values for Figure 7. Values are MAE-D with Top-1 AUROC percentages. The all-layer setting is the default configuration because it is strongest overall and avoids task-specific layer tuning.

Policy	Action-generation family	Attention depth	Main score	Head-selection settings	Evaluation role
🔗 OpenVLA	Latent-Readout / Autoregressive	32 layers / 32 heads	MAE-D Top-1	Top-1, Top-2, Top-4, Top-16	Discrete-action policy; supports token-statistic baselines
🔗 OpenVLA-OFT	Latent-Readout / Continuous OFT	32 layers / 32 heads	MAE-D Top-1	Top-1, Top-2, Top-4, Top-16	Same VLA family with continuous action decoding
🔗 QwenPI-Flow	Latent-Refinement / Flow-Style	36 layers / 40 heads	MAE-C Top-1	Top-1, Top-2, Top-4, Top-20	Cross-family flow-style policy

Table 10: Evaluation settings for MAE across the three VLA policies. The main score uses all layers with Top-1 head selection. Larger Top-k settings are reported only for the head-selection ablation; the half-head setting is Top-16 for 32-head OpenVLA-family policies and Top-20 for the 40-head QwenPI-Flow policy.

Policy	Input convention	Episode payload	Rendered prompt form
🔗 OpenVLA	Pure action-prompt format used by the OpenVLA policy.	One RGB observation image and the lower-cased LIBERO task instruction.	In: What action should the robot take to {instruction.lower()}? Out:
🔗 OpenVLA-OFT	OpenVLA-family action prompt with the OFT continuous-action decoder.	Primary image, optional wrist images when available, and the lower-cased task label.	In: What action should the robot take to {task_label.lower()}? Out:
🔗 QwenPI-Flow	Qwen3-VL multimodal message format followed by the StarVLA grounding request.	One or more image placeholders followed by the LIBERO instruction and object-localization request.	< im_start >user < vision_start >< image_pad >< vision_end > Your task is {instruction}. To identify the key objects for your task. Locate their bounding boxes in [x1,y1,x2,y2] format. < im_end > < im_start >assistant

Table 11: Model input templates used for policy conditioning. The table records only the prompt forms needed to reproduce the action-generation inputs; benchmark construction and experimental results are reported in the preceding appendix sections.

F Baseline Protocols and Applicability

This appendix defines the baselines used in Table 1 and clarifies their applicability to heterogeneous VLA action heads. We only discuss baselines that are part of the reported protocol. The main compatibility issue is whether a policy exposes autoregressive discrete action-token logits. Token-statistic baselines are meaningful for discrete OpenVLA, but they are not defined for OpenVLA-OFT’s continuous action head or QwenPI-Flow’s flow-matching action head without adding a separate likelihood model over executed actions. We keep the technically not-applicable rows visible in Table 1 because the interface mismatch is informative: conventional confidence baselines are often tied to a specific output parameterization, whereas MAE uses internal attention summaries from the policy forward process.

Random. Random assigns an i.i.d. uniform score to each episode and serves as a lower-bound ranking baseline under the same success/failure labels.

Verbal Confidence. Verbal Confidence adapts p(True)-style verbal self-checking to VLA evaluation. Since most VLA policies do not naturally output a calibrated verbal probability during action generation, we use an external multimodal evaluator as a proxy. For each episode, we query gpt-4.1 with the task instruction and a stitched contact-sheet image of sampled observations. The model returns a scalar confidence in $[0, 1]$, which is used directly as the episode-level reliability score. The evaluator is not given the ground-truth success label, simulator success state, object poses, perturbation metadata, robot state trajectories, generated action vectors, or oracle information.

Self-Consistency. Self-Consistency estimates reliability from repeated executions or repeated stochastic action predictions under the same task condition. A typical score is the agreement rate or success frequency across repeated rollouts. This baseline is a repeated-rollout estimator, so it is not equivalent to scoring a single deterministic execution after the fact. For OpenVLA-OFT and QwenPI-Flow, Table 1 therefore marks this estimator as technically not applicable to the single-rollout comparison protocol.

OpenVLA self-consistency adaptation. For OpenVLA, we group repeated executions by the same fixed initialization scenario. The reliability

score is

$$\text{SC}(s) = \frac{\# \text{ successful repeats under scenario } s}{\# \text{ repeats under scenario } s}.$$

A scenario is positive when the majority of its repeats succeed, while trial-level labels use each repeat’s individual success outcome. This baseline is only meaningful when matched-repeat groups are part of the evaluation protocol; the OpenVLA-OFT and QwenPI-Flow entries in Table 1 are single-rollout comparisons and therefore cannot instantiate the same estimator. This adaptation also exposes a limitation of self-consistency: repeated agreement can indicate a stable wrong behavior rather than task completion.

Maximum Softmax Probability. Maximum Softmax Probability measures the sharpness of autoregressive action-token predictions. For a generated action-token sequence of length T , with categorical distribution y_t over the vocabulary at generation step t , the episode score is

$$\frac{1}{T} \sum_{t=1}^T \max_i y_{t,i}.$$

This requires a vocabulary-level distribution for each generated action token, so it applies to discrete OpenVLA but not to continuous OFT or flow-matching PI action heads.

Perplexity. Perplexity-style confidence uses the negative log confidence of the selected action-token distribution:

$$\frac{1}{T} \sum_{t=1}^T -\log \left(\max_i y_{t,i} \right).$$

We invert the direction when necessary so that larger reported scores indicate higher estimated reliability. As with Maximum Softmax Probability, this score is only defined when action generation exposes token-level categorical probabilities.

Entropy. Entropy measures the uncertainty of the full output distribution:

$$\frac{1}{T} \sum_{t=1}^T \sum_i -y_{t,i} \log y_{t,i}.$$

Lower entropy indicates a sharper token distribution, so the reliability score uses the sign convention that larger is better. This is a token-distribution baseline and is not comparable for continuous action regression or flow integration without an additional probabilistic action model.

Verbal Confidence Prompt

System context: You are given a robot manipulation episode.

Image input: The attached image is a stitched contact sheet of observation frames sampled from the episode. Frames are ordered chronologically from left to right and top to bottom. Each frame is labeled with its timestep.

Task instruction:

{TASK_INSTRUCTION}

Evaluation request: Based on the task instruction and the attached contact-sheet image, estimate whether the robot successfully completed the task.

Output format: Return only one number between 0 and 1: 0 means definitely failed, 0.5 means uncertain, and 1 means definitely succeeded. Return only the number and nothing else.

Figure 9: Prompt used for the Verbal Confidence baseline. The stitched contact sheet is provided as an image input in the same multimodal API request.

Length-normalized Entropy. Length-normalized entropy uses top- k sampling to generate k candidate outputs $\mathcal{Y} = \{Y_1, \dots, Y_k\}$, then averages token entropy across the sampled outputs:

$$\frac{1}{k} \sum_{Y \in \mathcal{Y}} \frac{1}{T_Y} \sum_{t=1}^{T_Y} \sum_i -y_{t,i} \log y_{t,i}.$$

We set $k = 5$.

OpenVLA output-statistic values. We report reliability-oriented scores: Perplexity is represented by inverse PPL, Entropy by inverse token entropy, and Length-normalized Entropy by inverse length-normalized entropy, so larger values always indicate higher estimated reliability. These baselines can be computed for OpenVLA because it generates discrete action tokens with vocabulary-level probabilities; applying them to OpenVLA-OFT or QwenPI-Flow would require adding a separate likelihood model over continuous executed actions.

Applicability to heterogeneous VLAs. Discrete OpenVLA exposes autoregressive action-token distributions, so token-probability baselines such as Maximum Softmax Probability, Perplexity, Entropy, and Length-normalized Entropy are conceptually defined for that model family. OpenVLA-OFT changes the action interface: it keeps the OpenVLA backbone but predicts continuous actions through a regression head, so the placeholder action slots are not generated action tokens and their logits do not define the executed action. QwenPI-Flow uses a flow-matching action head that maps visual-language hidden states and noise through iterative velocity prediction; its output is a

continuous action trajectory rather than a categorical token sequence. Applying token-probability baselines to these models would evaluate the wrong random variable. By contrast, **MAE** reads internal attention entropy during the same policy forward process and then uses architecture-aware aggregation, **MAE-D** for latent-readout policies and **MAE-C** for latent-refinement policies, which is why it remains comparable across the heterogeneous action heads in Table 1.

Component	Detail
GPU Hardware	NVIDIA H100
Model Evaluated	QwenPI-Flow
Avg. Rollout Time	~14.0 s / episode
Avg. MAE Extra Time	0.57 s / episode
MAE Latency Overhead	4.09% ($< 0.1 \times$)
Memory Overhead	Negligible (Reuses internal attention)

Table 12: Empirical cost analysis of **MAE**. The overhead strictly satisfies the $< 0.1 \times$ boundary highlighted in the Pareto-optimal zone of Figure 6.

G Model Input Templates

Table 11 reports the policy input formats used to condition the three evaluated VLA backbones. The OpenVLA-family policies use the same action-query form with lower-cased LIBERO instructions. QwenPI-Flow uses a Qwen3-VL-style multimodal message and appends the grounding text expected by the StarVLA policy. We place these templates after the experimental tables because they are protocol details rather than additional results.

H Dataset-source considerations.

A potential concern is that a reliability score may separate standard LIBERO episodes from LIBERO-PRO episodes rather than estimate episode-level action reliability. LIBERO-Reflect is designed to reduce this first-order source shortcut at both the construction and evaluation levels. First, the two nominal pools are matched by suite organization, task count, number of initializations, simulator, policy interface, and rollout protocol. Thus, differences in evaluation code, control horizon, observation logging, and metric computation are not available as cues to the scoring function. Second, the benchmark does not inherit labels from dataset membership. The nominal source is used only to assemble a mixed-difficulty evaluation pool; all reported metrics are computed using the realized simulator success flag of each rollout. As a result, failures from the standard LIBERO side and successes from the LIBERO-PRO side are retained and evaluated according to their actual outcomes. This distinction is important for interpreting MAE. A source-level shortcut would assign nearly uniform reliability to all standard LIBERO episodes and uniformly low reliability to all LIBERO-PRO episodes. Such a rule is penalized whenever nominal source and realized outcome disagree, and it does not capture within-source variation among episodes with the same dataset origin but different execution outcomes. In contrast, MAE is computed from the policy’s internal attention dynamics during the same forward passes that generate actions, after conditioning on the current visual observation and language instruction. The score therefore has no access to split identifiers or nominal source labels; it can only exploit how the latent action state routes information under the current episode condition.

We use LIBERO-PRO as a controlled source of challenging rollouts to increase the density of failures needed for reliability ranking, not as a substitute for failure labels. The resulting benchmark should therefore be understood as an episode-level reliability test under a matched mixed-difficulty pool. While this construction mitigates first-order dataset-source shortcuts, it does not claim to remove all distributional differences between standard and PRO scenes. This is why we report actual-outcome labels and diagnostic success rates for both nominal pools, and interpret the results as reliability ranking over realized executions rather than

as source-invariant classification.

I Architecture-Determined Entropy Orientation

We further clarify that the opposite entropy orientations used by MAE-D and MAE-C are not chosen post hoc from test-set performance. They are determined by the action-generation interface of the evaluated policy before any episode-level reliability metric is computed. The distinction follows from the role played by the visual-attention distribution in the transition kernel of the conditional generative Markov chain.

For Latent-Readout VLAs, the internal generation process first builds a latent action representation and then maps the final latent state to an executable action through a readout or decoder. In this family, the transition kernel must progressively consolidate task-relevant visual evidence into the latent state before the final readout. A reliable transition therefore tends to concentrate action-query attention on the relevant object, region, or spatial relation needed for the action. Conversely, diffuse visual addressing indicates that the latent action state has not localized the necessary visual evidence, which often corresponds to ambiguous grounding or incorrect object-action binding. For this reason, lower visual entropy is assigned higher reliability, yielding the decreasing-entropy orientation MAE-D.

For Latent-Refinement VLAs, the policy maintains an explicit action or trajectory variable and repeatedly refines it under visual-language conditioning, as in flow- or denoising-style generation. Here the transition kernel is not merely preparing a single terminal readout; it repeatedly corrects an evolving action trajectory by re-querying the condition. Reliable refinement must therefore preserve sufficient visual coverage during the refinement process, especially when the correct action depends on object relations, spatial constraints, or multi-step manipulation context. Prematurely low visual entropy can indicate early collapse to a narrow visual region before the action trajectory has incorporated all relevant constraints. Thus, in this family, higher visual entropy reflects sustained visual exploration during refinement and is assigned higher reliability, yielding the increasing-entropy orientation MAE-C. This orientation rule is architecture-level rather than data-fitted. A model is assigned to the Latent-Readout family if its action is produced by decoding or reading out a final latent state with-

out iterative refinement of an explicit action variable. It is assigned to the Latent-Refinement family if it maintains an action or trajectory representation that is updated across multiple refinement or denoising steps under repeated conditioning. This decision can be made from the model’s inference computation graph alone and does not require success/failure labels, external supervision, or validation-set optimization. Accordingly, **MAE-D** is fixed for Latent-Readout policies and **MAE-C** is fixed for Latent-Refinement policies before evaluating AUROC, AUPR, or FPR@95.

The same principle also separates orientation from head and layer aggregation. The Top-1 operation used in the main protocol is a deterministic per-episode reduction over oriented entropy values, not a fixed attention head selected by test labels. Likewise, all-layer aggregation is used as the default rule to avoid selecting a task-specific or model-specific layer band. The Top- k and layer-band experiments are therefore sensitivity analyses of a frozen scoring rule rather than procedures for choosing the reported orientation.

J Cost Analysis Details

To substantiate the efficiency claims in Section 4, we profile the wall-clock latency of **MAE** during evaluation. Table 12 reports the average execution time per episode and the extra latency incurred by **MAE**. The overhead is strictly bounded because **MAE** reuses the internal attention matrices from the same forward pass and only performs entropy computation and head aggregation.